TECNOLÓGICO
NACIONAL DE MÉXICO

Proyecto Fase 3: Ejecución - Codificación del Modelo

Alumnos:
Ricardo Alan Sanchez Alonso
Flores Hernández Luis Alberto
Carlos Damian Suarez Garcia

Desarrollo: Para la fase número 3 se tomaron en cuenta las clases y métodos descritos en las fases 1 y 2 del proyecto y programando dentro de VS Code:

Spawtrap > src > Main.java > Main > main(String[])

```java
package src;

import java.util.Scanner;

public class Main {
    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println(x: "=== Bienvenido a Spawntrap ===");

        System.out.print(s: "Ingresa tu nombre de usuario: ");
        String nombre = sc.nextLine();

        CuentaUsuario usuario = new CuentaUsuario(nombre, id: 1);

        int opcion;

        do {
            System.out.println(x: "\n--- MENU ---");
            System.out.println(x: "1. Ver perfil");
            System.out.println(x: "2. Agregar puntos");
            System.out.println(x: "3. Crear reseña");
            System.out.println(x: "4. Ver inventario");
            System.out.println(x: "5. Salir");
            System.out.print(s: "Elige una opción: ");

            opcion = sc.nextInt();
            sc.nextLine(); // limpiar buffer

            switch (opcion) {

                case 1:
                    usuario.mostrarPerfil();
                    break;
```

```java
package src;

public class CuentaUsuario {
    private String nombreUsuario;
    private int cantidadPuntos;
    private int id;
    private Inventario inventario;

    public CuentaUsuario(String nombreUsuario, int id) {
        this.nombreUsuario = nombreUsuario;
        this.id = id;
        this.cantidadPuntos = 0;
        this.inventario = new Inventario();
    }

    public void agregarPuntos(int puntos) {
        cantidadPuntos += puntos;
    }

public void restarPuntos(int puntos) {

    if (puntos > cantidadPuntos) { // if
        System.out.println(x: "No tienes suficientes puntos.");
    } else {
        cantidadPuntos -= puntos;
        System.out.println("Puntos restantes: " + cantidadPuntos);
    }
    }

    public void crearResena(Juego juego, String texto, int puntuacion) {
        Resena resena = new Resena(this, juego, texto, puntuacion);
        resena.mostrarResena();
    }

    public void mostrarPerfil() {
        System.out.println(this);
    }
```

```java
package src;

public class Juego {

    private String titulo;
    private int idJuego;
    private String desarrollador;

    public Juego(String titulo, int idJuego, String desarrollador) {

        if (titulo == null || titulo.trim().isEmpty()) {
            throw new IllegalArgumentException(s: "El título no puede estar vacío.");
        }

        if (idJuego <= 0) {
            throw new IllegalArgumentException(s: "El ID debe ser mayor a 0.");
        }

        if (desarrollador == null || desarrollador.trim().isEmpty()) {
            throw new IllegalArgumentException(s: "El desarrollador no puede estar vacío.");
        }

        this.titulo = titulo;
        this.idJuego = idJuego;
        this.desarrollador = desarrollador;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getIdJuego() {
        return idJuego;
    }

    public String getDesarrollador() {
        return desarrollador;
```

```java
package src;

public class Item {
    protected String nombre;
    protected String tipo;

    public Item(String nombre, String tipo) {
        this.nombre = nombre;
        this.tipo = tipo;
    }

public void usarItem() {

    switch (tipo) { // switch

        case "Cromo":
            System.out.println(x: "Este cromo es coleccionable.");
            break;

        case "Consumible":
            System.out.println(x: "Usando consumible...");
            break;

        default:
            System.out.println(x: "Tipo de item desconocido.");
    }
    }

    @Override
    public String toString() {
        return "Item: " + nombre + " | Tipo: " + tipo;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
                    usuario.getInventario().agregarItem(
                    new Cromo(nombre: "Cromo épico", juegoTest, rareza: "Épico")
                    );

                    // Mostrar inventario
                    usuario.getInventario().mostrarInventario();

                     // Agregar item consumible
                    Item item = new Item(nombre: "Poción", tipo: "Consumible");
                    usuario.getInventario().agregarItem(item);

                    // Probar el switch
                    item.usarItem();

                    break;
                case 5:
                    System.out.println(x: "Saliendo de Spawntrap...");
                    break;

                default:
                    System.out.println(x: "Opción inválida.");
            }

        } while (opcion != 5);

        sc.close();
    }


}
```

```java
package src;

public class Cromo extends Item {
    private Juego juego;
    private String rareza;

    public Cromo(String nombre, Juego juego, String rareza) {
        super(nombre, tipo: "Cromo");
        this.juego = juego;
        this.rareza = rareza;
    }

    public void intercambiar(CuentaUsuario usuario) {
        System.out.println("Intercambiando con: " + usuario);
    }

    @Override
    public String toString() {
        return "Cromo: " + nombre +
                " | Juego: " + juego.getTitulo() +
                " | Rareza: " + rareza;
    }
}
```

```java
public class Juego {
    public Juego(String titulo, int idJuego, String desarrollador) {
        this.idJuego = idJuego;
        this.desarrollador = desarrollador;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getIdJuego() {
        return idJuego;
    }

    public String getDesarrollador() {
        return desarrollador;
    }

    @Override
    public String toString() {
        return String.format(format: "Juego: %s | ID: %d | Dev: %s",
                titulo, idJuego, desarrollador);
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

            case 2:
                System.out.print(s: "¿Cuántos puntos agregar?: ");
                int puntos = sc.nextInt();
                usuario.agregarPuntos(puntos);
                System.out.println(x: "Puntos agregados correctamente.");
                break;

            case 3:
              String titulo = sc.nextLine();

                try {
                    Juego juego = new Juego(titulo, idJuego: 1, desarrollador: "Desconocido");

                    System.out.print(s: "Escribe tu reseña: ");
                    String texto = sc.nextLine();

                    System.out.print(s: "Puntuación (1-10): ");
                    int puntuacion = sc.nextInt();

                    usuario.crearResena(juego, texto, puntuacion);

                } catch (IllegalArgumentException e) {
                        System.out.println("Error al crear el juego: " + e.getMessage());
                }

            case 4:

                // Crear juego de prueba
                Juego juegoTest = new Juego(titulo: "Juego Demo", idJuego: 99, desarrollador: "DevTest");

                // Agregar cromo
                usuario.getInventario().agregarItem(
                new Cromo(nombre: "Cromo épico", juegoTest, rareza: "Épico")
                );
```

```java
public void restarPuntos(int puntos) {
    } else {
        cantidadPuntos -= puntos;
        System.out.println("Puntos restantes: " + cantidadPuntos);
    }
    }

    public void crearResena(Juego juego, String texto, int puntuacion) {
        Resena resena = new Resena(this, juego, texto, puntuacion);
        resena.mostrarResena();
    }

    public void mostrarPerfil() {
        System.out.println(this);
    }

    public Inventario getInventario() {
        return inventario;
    }

    @Override
    public String toString() {
        return "Usuario: " + nombreUsuario +
                " | ID: " + id +
                " | Puntos: " + cantidadPuntos;
    }
}
```

```java
package src;

import java.util.ArrayList;

public class Inventario {
    private ArrayList<Item> listaItems;

    public Inventario() {
        listaItems = new ArrayList<>();
    }

    public void agregarItem(Item item) {
        listaItems.add(item);
    }

    public void eliminarItem(Item item) {
        listaItems.remove(item);
    }

    public void mostrarInventario() {

    if (listaItems.isEmpty()) { // if
        System.out.println(x: "Inventario vacío.");
    } else {
        System.out.println(x: "=== Inventario ===");

        for (int i = 0; i < listaItems.size(); i++) { // for
            System.out.println((i + 1) + ". " + listaItems.get(i));
        }
    }
}
}
```

Spawtrap > src > Logro.java > Logro

```java
package src;

public class Logro {
    private String nombre;
    private String descripcion;
    private int puntos;

    public Logro(String nombre, String descripcion, int puntos) {
        this.nombre = nombre;
        this.descripcion = descripcion;
        this.puntos = puntos;
    }

    @Override
    public String toString() {
        return "Logro: " + nombre +
                " | " + descripcion +
                " | Puntos: " + puntos;
    }
}
```

```java
package src;

public class Resena {
    private CuentaUsuario autor;
    private Juego juego;
    private String texto;
    private int puntuacion;

    public Resena(CuentaUsuario autor, Juego juego, String texto, int puntuacion) {
        this.autor = autor;
        this.juego = juego;
        this.texto = texto;
        this.puntuacion = puntuacion;
    }

    public void mostrarResena() {
        System.out.println(this);
    }

    @Override
    public String toString() {
        return "Reseña de " + juego.getTitulo() +
                " | Puntuación: " + puntuacion +
                " | " + texto;
    }
}
```

En dicho código podemos ver declaradas cada una de las clases y sus atributos, siendo estas funcionales, la complejidad del mismo recae en la implementación de ciclos while/for y if/switch y los distintos casos que se generan con los mismos, aplicando lo siguiente:

Manejo de Excepciones (try-catch y validación):

En la clase Juego se implementan validaciones estrictas (if (titulo == null...)) que lanzan un IllegalArgumentException si los datos son incorrectos. Esto se conecta con el Main donde se usa un bloque try-catch para intentar instanciar el juego y capturar el error si ocurre, evitando que el programa se cierre abruptamente y mostrando el mensaje de error al usuario.

Uso de Colecciones Genéricas y Composición (ArrayList<Item>):

La clase Inventario no usa arreglos estáticos, sino un ArrayList<Item>. Lo que lleva a una relación de *composición* (una CuentaUsuario *tiene un* Inventario). Además, al usar la clase padre Item en el diamante <>, permite almacenar cualquier clase hija (como Cromo) gracias al polimorfismo.

Lógica del Menú Interactivo y Gestión de Estado (do-while + switch):

En el Main se usó la combinación del bucle do-while con la estructura switch que mantiene la ejecución del programa activa y el estado del objeto usuario en memoria de forma persistente mientras se navega por el menú, procesando la entrada del Scanner sin perder la sesión.

Relaciones de Asociación y Referencias a Objetos (Resena.java): La mayor complejidad estructural aquí radica en cómo se enlazan las diferentes entidades del ecosistema. La clase Resena actúa como una entidad asociativa. En lugar de recibir simples tipos de datos primitivos o *Strings*, su constructor exige referencias directas a objetos complejos ya existentes en memoria: el autor (tipo CuentaUsuario) y el juego (tipo Juego).

Evidencia de IA: Para el siguiente pasó se presentó el siguiente Promp a ChatGPT para la optimización del código y sua adaptación a una interfaz funcional:

✅ Versión mejorada (Clean Code + excepciones)

Java

```java
package src;

public class Juego {

    private String titulo;
    private int idJuego;
    private String desarrollador;

    public Juego(String titulo, int idJuego, String desarrollador) {

        if (titulo == null || titulo.trim().isEmpty()) {
            throw new IllegalArgumentException("El título no puede estar vacío.");
        }

        if (idJuego <= 0) {
            throw new IllegalArgumentException("El ID debe ser mayor a 0.");
        }

        if (desarrollador == null || desarrollador.trim().isEmpty()) {
            throw new IllegalArgumentException("El desarrollador no puede estar vacío.");
        }

        this.titulo = titulo;
        this.idJuego = idJuego;
        this.desarrollador = desarrollador;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getIdJuego() {
        return idJuego;
    }
```

Dictat Usar Voz Ctrl + Alt + V

```java
    }

    public String getTitulo() {
        return titulo;
    }

    public int getIdJuego() {
        return idJuego;
    }

    public String getDesarrollador() {
        return desarrollador;
    }

    @Override
    public String toString() {
        return String.format("Juego: %s | ID: %d | Dev: %s",
                titulo, idJuego, desarrollador);
    }
}
```

🧠 ¿Por qué estos cambios mejoran tu programa?

1️⃣ Validaciones → evitan errores futuros

Antes:

Java

```java
new Juego("", -1, "");
```

✔ Se creaba sin problema ❌ (mal diseño)

Ahora:

Java

```java
throw new IllegalArgumentException(...)
```

Dictat Usar Voz Ctrl + Alt + V

```
throw new IllegalArgumentException(...)
```

- ✔ Evitas datos inválidos desde el inicio
- ✔ Detectas errores más rápido

👉 Esto se llama **fail fast (fallar rápido)**

2️⃣ Uso de excepciones (mejor control)

Usamos:

```Java
IllegalArgumentException
```

- ✔ Es estándar en Java
- ✔ Indica claramente que el error es por datos incorrectos

3️⃣ Mejor legibilidad (Clean Code)

Antes:

```Java
return "Juego: " + titulo + ...
```

Ahora:

```Java
String.format(...)
```

- ✔ Más limpio
- ✔ Más fácil de mantener

Dictate Usar Voz Ctrl + Alt + V

Agregamos:

```Java
getIdJuego()
getDesarrollador()
```

- ✔ Permite acceder a los datos de forma controlada
- ✔ Mejora reutilización del código

⚠️ IMPORTANTE (para que no truene tu programa)

Como ahora hay excepciones, debes manejarlas en `Main.java`:

```Java
try {
    Juego juego = new Juego("Elden Ring", 1, "FromSoftware");
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
```

🧠 Resultado final

Con estos cambios tu código ahora es:

- ✔ Más seguro
- ✔ Más profesional
- ✔ Más fácil de mantener
- ✔ Evita errores desde el inicio
- ✔ Sigue principios de Clean Code

Dictate Usar Voz Ctrl + Alt + V

Estado de Avance:

Conclusión

En esta fase, el equipo tomó los diseños y diagramas creados anteriormente y los convirtió en código Java real y funcional. Cada clase planteada desde el inicio, como el usuario, los juegos, el inventario y las reseñas, fue programada correctamente con sus atributos y métodos correspondientes.

El programa en esta fase ahora es capaz de ejecutarse e interactuar con el usuario a través de un menú, permitiendo acciones como agregar puntos, crear reseñas y gestionar un inventario. Además, se aplicaron buenas prácticas de programación como el manejo de errores mediante bloques try-catch, el uso de colecciones genéricas con ArrayList<Item> y estructuras de control do-while.

Como apoyo adicional, se utilizó inteligencia artificial para mejorar y optimizar el código, obteniendo un resultado más ordenado y fácil de mantener. En resumen, esta fase cambió el punto donde el proyecto dejó de ser solo una idea en papel a programa funcional.